# Closed-Form Discovery of PDE Solutions via Compositional EML Primitives

Lee Mosbacker
Caprica AI Inc.
lee.mosbacker@gmail.com

## Abstract

We present a method for recovering exact closed-form solutions to partial differential equations from data. The core technical contributions are twofold. First, a *normalization pipeline* that resolves gauge freedoms in the parameterization of composed elementary functions—absorbing exponential biases into multiplicative coefficients and canonicalizing trigonometric phases—enabling reliable snapping of learned parameters to exact symbolic values. Second, a *single-term search strategy* that trains each candidate expression in isolation, avoiding the signal-splitting failure mode of multi-term models with sparsity regularization.

These techniques operate within a structured ansatz we call the COMPOSEHEAD, which represents candidate solutions as products of axis-aligned elementary functions—matching the separable structure common to analytical PDE solutions. The primitive basis $\{\sin, \cos, \exp, \ln\}$ is grounded in the EML (Exp-Minus-Log) universality theorem of Odrzywołek Odrzywołek (2026), which provides a principled justification for this function library and guarantees closure under differentiation—a property essential for computing PDE residuals within the compositional graph.

We demonstrate the approach on benchmarks spanning the 1-D heat equation, 1-D wave equation, the non-separable traveling wave $\sin(x - 2t)$, and the two-dimensional Navier–Stokes equations, recovering every target at machine precision (pointwise error $< 10^{-7}$). Most notably, we show that the pipeline can operate in a *PDE-residual-only* mode—receiving no solution data, only the governing equations and initial conditions—and still discovers the exact Taylor–Green vortex solution $u = \sin(x)\cos(y)\exp(-t/5)$, $v = -\cos(x)\sin(y)\exp(-t/5)$, functioning as a symbolic physics-informed neural network (PINN) whose output is a verifiable closed-form expression rather than an opaque weight matrix. A systematic screening of 13 three-dimensional Navier–Stokes initial conditions reveals that vortex stretching prevents exact exponential solutions for all non-Beltrami flows, while Beltrami (ABC) flows are recovered exactly—providing a computational explanation for the scarcity of known 3-D exact solutions. Ablation studies confirm that normalization is the most impactful design choice, improving snap accuracy by five orders of magnitude.

## 1 Introduction

Partial differential equations govern phenomena across fluid dynamics, quantum mechanics, and continuum mechanics, yet obtaining analytical solutions remains one of the most difficult tasks in applied mathematics. When closed-form solutions do exist, they provide qualitative insight, asymptotic guarantees, and computational efficiency that no numerical approximation can match. The recent success of deep-learning-based PDE solvers has dramatically expanded the class of problems that can be tackled computationally, but it has done so at the cost of interpretability: a trained physics-informed neural network (PINN) Raissi et al. (2019) produces a set of floating-point weights from which no human-readable formula can be extracted.

**Neural PDE solvers: accuracy without insight.** PINNs and their descendants embed the governing PDE into the loss function of a neural network and optimize over the network parameters so that the learned map approximately satisfies the differential equation together with its boundary and initial conditions. This paradigm is flexible—it requires neither a mesh nor a specialized numerical scheme—and has yielded impressive results on forward and inverse problems alike. However, the output of such a solver is an opaque parameterization: one obtains a weight matrix, not a formula.

Post-hoc interpretability techniques can highlight which input regions matter, but they cannot recover the symbolic structure that a mathematician would recognize as a solution.

**Symbolic regression: powerful but limited in scope.** An alternative line of work seeks to discover symbolic expressions directly from data. Tools such as PySR Cranmer (2023) and AI Feynman Udrescu and Tegmark (2020) have achieved striking success on algebraic physical laws—rediscovering conservation laws, constitutive relations, and dimensionless groupings from noisy measurements. These methods search over a grammar of elementary operations and fit scalar expressions of moderate complexity. Yet PDE solutions pose a qualitatively different challenge. A typical analytical solution is not a single algebraic expression in one variable but a *product of functions of different variables*—for instance, $\sin(x)\cos(y)\exp(-\nu t)$—and the combinatorial explosion of multi-variable compositions quickly overwhelms generic symbolic regression engines.

**Elementary functions as a principled basis.** A natural question arises: which set of primitive functions should a symbolic search use? An ad hoc selection risks omitting functions needed for a given solution class. The recent universality result of Odrzywołek Odrzywołek (2026) provides a principled answer. Odrzywołek proved that the two-argument function $\mathrm{eml}(x, y) = e^x - \ln y$ is *universal*: every computable function can be represented as a finite composition of $\mathrm{eml}$ with itself. Although the raw EML representation is unwieldy—multiplication alone requires a reverse-Polish representation of length 41—the result has a practically important corollary: the elementary functions $\{\sin, \cos, \exp, \ln\}$ can each be constructed from EML compositions via algebraic identities. This establishes the set $\{\sin, \cos, \exp, \ln\}$ not as an arbitrary library but as a *verified basis* derived from a universal primitive. Moreover, this function class is closed under partial differentiation, a property that is essential when the loss function involves PDE residuals computed via automatic differentiation.

**Our approach: normalization-driven symbolic recovery.** The central challenge in recovering exact symbolic expressions from gradient-trained models is not the choice of architecture but rather the *extraction* of clean symbolic forms from continuously parameterized representations. Elementary functions exhibit gauge freedoms that confound naive coefficient readout: a trained model may represent $\cos(x)$ as $\sin(x+\pi/2)$, absorb constant factors into exponential biases, or split a single physical term across multiple basis functions with nearly cancelling coefficients. We address these difficulties with two key ideas.

First, we introduce a *normalization pipeline* that resolves the gauge freedom inherent in parameterized elementary functions. The pipeline absorbs exponential biases into linear coefficients, canonicalizes trigonometric phases by exploiting the identities $\sin(x + \pi/2) = \cos(x)$ and $\cos(x + \pi) = -\cos(x)$, and rounds the resulting coefficients to nearby rational or otherwise recognizable values. This normalization step is the primary enabler of exact symbolic recovery, improving coefficient accuracy by approximately five orders of magnitude relative to direct readout of trained parameters.

Second, we adopt a *single-term search strategy* in which the model is trained with $K = 1$ active term at a time, avoiding the signal-splitting pathology in which a multi-term ansatz distributes a single physical mode across several basis functions with partially cancelling coefficients. Multi-term solutions are assembled by sequential residual fitting.

These ideas are realized in a differentiable architecture we call the COMPOSEHEAD, which represents candidate solutions as trainable linear combinations of *separable terms*:

$$\hat{u}(\mathbf{x}) \;=\; \sum_{k=1}^{K} c_k \prod_{d=1}^{D} f_{k,d}\big(\alpha_{k,d}\, x_d + \beta_{k,d}\big), \tag{1}$$

with each factor $f_{k,d}$ drawn from the primitive library $\{\sin, \cos, \exp, \ln, \mathrm{id}\}$ and each $\alpha_{k,d}$, $\beta_{k,d}$, $c_k$ a trainable scalar coefficient. The separable product structure mirrors the classical method of separation of variables, which has been a cornerstone of analytical PDE theory for three centuries. By restricting the hypothesis class to this well-understood form, the optimization problem reduces from learning arbitrary neural-network weights to identifying which elementary functions compose along each coordinate axis and with what coefficients.

**Scope and limitations.** The method proposed here is best understood as *symbolic regression specialized to separable PDE solutions*. The ansatz in Equation (1) is expressive for the broad class of problems whose analytical solutions factor across coordinate dimensions—a class that includes many canonical problems in mathematical physics—but it does not address existence, uniqueness, or regularity questions, nor does it replace mesh-based solvers for problems lacking closed-form solutions. Problems whose solutions are inherently non-separable (e.g., those requiring similarity variables or integral representations) fall outside the current scope. Our claims are accordingly conservative: we demonstrate that, when an analytical solution exists and has separable structure, the method can recover it exactly from PDE-residual supervision alone.

**Contributions.** The principal contributions of this work are as follows.

- **Normalization and snapping pipeline.** We introduce a post-training procedure that resolves the gauge freedom in parameterized elementary functions—absorbing exponential biases, canonicalizing trigonometric phases, and snapping coefficients to exact values—converting a continuously parameterized model into a closed-form symbolic expression. This pipeline yields approximately five orders of magnitude improvement in coefficient accuracy relative to direct parameter readout.
- **Single-term search strategy.** We show that training with one active term at a time, followed by sequential residual fitting, avoids the signal-splitting pathology that arises in multi-term optimization and is essential for reliable symbolic extraction.
- **ComposeHead architecture with separable ansatz.** We introduce a differentiable architecture whose hypothesis class is the set of linear combinations of separable products of axis-aligned elementary functions, enabling gradient-based search over symbolic PDE solution candidates.
- **EML-grounded primitive basis.** We provide differentiable, algebraically verified implementations of $\exp$, $\ln$, $\sin$, and $\cos$ built from compositions of the EML function of Odrzywołek (2026), establishing a principled (rather than ad hoc) justification for the primitive library and ensuring differentiation closure of the function class.
- **Exact recovery across PDE classes.** We demonstrate the pipeline on four PDE benchmarks—1-D heat, 1-D wave, the non-separable traveling wave $\sin(x - 2t)$, and the 2-D incompressible Navier–Stokes equations—recovering every target at machine precision (MAE $< 10^{-7}$).
- **PDE-residual-only symbolic discovery.** We show that the pipeline can discover exact analytical solutions from the governing equations and initial conditions alone, with no solution data whatsoever. Applied to the 2-D Navier–Stokes equations, it recovers the Taylor–Green vortex $u = \sin(x)\cos(y)\exp(-t/5)$, $v = -\cos(x)\sin(y)\exp(-t/5)$ at MAE $5 \times 10^{-8}$—functioning as a symbolic PINN whose output is a verifiable closed-form expression rather than an opaque weight matrix.

The remainder of the paper is organized as follows. Section 2 reviews the EML universality result and the relevant symbolic regression literature. Section 3 details the primitive constructions, the ComposeHead architecture, and the normalization pipeline. Section 4 presents the Taylor–Green vortex experiment and ablation studies. Section 5 discusses limitations and directions for future work.

## 2 Background

This section introduces the EML function and its universality properties, describes the verified primitive constructions that form the basis of our method, and reviews the two families of prior work—physics-informed neural networks and symbolic regression—against which we position our approach.

### 2.1 The EML Function

The *Exp-Minus-Log* (EML) function is defined as

$$\mathrm{eml}(x, y) \;=\; e^x - \ln y, \qquad x \in \mathbb{R},\; y > 0. \tag{2}$$

Odrzywołek Odrzywołek (2026) established that $\mathrm{eml}$ is a *universal function*: every computable function admits a representation as a finite recursive composition of $\mathrm{eml}$ operations. This result is surprising in that a single bivariate primitive suffices to generate the full hierarchy of elementary and special functions.

Several elementary identities follow directly from the definition:

$$\mathrm{eml}(x, 1) = e^x, \qquad \text{since } \ln 1 = 0, \tag{3}$$

$$\mathrm{eml}(0, y) = 1 - \ln y, \qquad \text{since } e^0 = 1, \tag{4}$$

$$\frac{\partial\, \mathrm{eml}}{\partial x} = e^x, \qquad \frac{\partial\, \mathrm{eml}}{\partial y} = -\frac{1}{y}. \tag{5}$$

The closure of $\mathrm{eml}$ under differentiation—each partial derivative is itself expressible via the primitives $e^x$ and $1/y$—is a property we exploit heavily in Section 3, where PDE residuals must be differentiated through the compositional graph.

**Limitations.** The universality claim is subject to important caveats. Smith Smith (2024) demonstrated that $\mathrm{eml}$ cannot express roots of generic quintic polynomials whose associated Galois groups are non-solvable, a consequence of the Abel–Ruffini theorem applied to the monodromy structure of $\mathrm{eml}$ compositions. Separately, İpek Ipek (2024) proposed an extension of the universality result (arXiv:2604.13871), but the paper was subsequently withdrawn by the author citing a "fundamental limitation" in the proof. These results constrain the theoretical scope of EML universality but do not affect the practical utility of the framework for compositions of elementary functions—the regime relevant to PDE solutions encountered in physics.

**Depth requirements.** Even for elementary operations, the depth of the required $\mathrm{eml}$ composition can be substantial. Odrzywołek Odrzywołek (2026) reports that expressing $\ln$ requires depth 3, multiplication requires a reverse-Polish notation (RPN) representation of length 41, and the constant $\pi$ requires length 193 or more. These figures render direct gradient-based optimization over raw EML expression trees impractical: the search space is combinatorially vast, and the resulting computation graphs are deep enough to suffer from vanishing and exploding gradients. This observation motivates the strategy described in Section 2.2: rather than learning $\mathrm{eml}$ trees, we pre-compute algebraically verified constructions for a small set of elementary functions and optimize over compositions of those constructions.

### 2.2 Verified Primitive Constructions

The central idea of our method is to bridge the gap between the theoretical universality of $\mathrm{eml}$ and the practical requirements of gradient-based optimization. We achieve this by pre-computing a library of fixed, algebraically verified constructions that express standard elementary functions as $\mathrm{eml}$ compositions. Each construction is implemented as a frozen PyTorch module with no learnable parameters; the optimization problem is thus lifted from the space of raw $\mathrm{eml}$ trees to the space of linear combinations of these verified primitives.

**Exponential.** The simplest construction is immediate from (3):

$$\exp(x) = \mathrm{eml}(x, 1). \tag{6}$$

This is a depth-1 composition.

**Natural logarithm.** The logarithm requires depth 3. We construct it as follows:

$$\ln(z) = \mathrm{eml}\Big(1,\ \mathrm{eml}\big(\mathrm{eml}(1, z),\ 1\big)\Big). \tag{7}$$

To verify, evaluate from the inside outward. First, $\mathrm{eml}(1, z) = e - \ln z$. Second, $\mathrm{eml}(e - \ln z,\ 1) = \exp(e - \ln z)$, since $\ln 1 = 0$. Third,

$$\mathrm{eml}\big(1,\ \exp(e - \ln z)\big) = e - \ln\big(\exp(e - \ln z)\big) = e - (e - \ln z) = \ln z.$$

**Sine and cosine.** The trigonometric functions are obtained via the complex extension of $\mathrm{eml}$. Because $\mathrm{eml}(ix, 1) = \exp(ix) - \ln 1 = \exp(ix)$, Euler's formula gives

$$\sin(x) = \mathrm{Im}\big(\mathrm{eml}(ix, 1)\big), \tag{8}$$

$$\cos(x) = \mathrm{Re}\big(\mathrm{eml}(ix, 1)\big). \tag{9}$$

In practice, we implement $\mathrm{eml}(ix, 1)$ using PyTorch's complex tensor arithmetic, extracting the real and imaginary parts to obtain $\cos$ and $\sin$ respectively.

**The constant $\pi$.** The circle constant is recovered from the complex logarithm construction:

$$\pi = -\,\mathrm{Im}\big(\ln(-1)\big), \tag{10}$$

where $\ln$ is the depth-3 EML construction of (7) extended to complex arguments.

**Verification protocol.** It is essential to emphasize that none of these constructions are learned. They are algebraic identities, proved symbolically and implemented as deterministic modules. We verify each implementation numerically against the corresponding PyTorch intrinsic (`torch.sin`, `torch.cos`, `torch.exp`, `torch.log`) over $10^6$ uniformly sampled points, confirming pointwise agreement to within $10^{-5}$. This verification step guarantees that the compositional basis inherits the full numerical fidelity of the underlying hardware transcendentals.

### 2.3 Physics-Informed Neural Networks

Physics-informed neural networks (PINNs), introduced by Raissi, Perdikaris, and Karniadakis Raissi et al. (2019), embed the governing PDE directly into the training loss. Given a PDE of the form $\mathcal{N}[u] = 0$ with boundary conditions $\mathcal{B}[u] = 0$, a PINN parameterizes the solution as a neural network $u_\theta$ and minimizes

$$\mathcal{L} = \underbrace{\frac{1}{N_d}\sum_{i=1}^{N_d}\big|u_\theta(\mathbf{x}_i) - u_i^{\mathrm{data}}\big|^2}_{\mathcal{L}_{\mathrm{data}}} + \lambda\ \underbrace{\frac{1}{N_r}\sum_{j=1}^{N_r}\big|\mathcal{N}[u_\theta](\mathbf{x}_j)\big|^2}_{\mathcal{L}_{\mathrm{PDE}}}, \tag{11}$$

where $\{\mathbf{x}_i, u_i^{\mathrm{data}}\}$ are observed data points, $\{\mathbf{x}_j\}$ are collocation points sampled in the interior of the domain, and $\lambda$ is a weighting hyperparameter.

PINNs possess several attractive properties: they are mesh-free, accommodate irregular geometries naturally, and can incorporate partial or noisy observations. These features have driven widespread adoption across computational physics. However, for the purposes of the present work, PINNs suffer from a fundamental limitation: their output is a set of network weights $\theta$, not a symbolic expression. Consequently, PINN solutions offer no direct interpretability, cannot be analytically verified against the governing equations, and do not transfer to related problems in the way that a closed-form expression does. Our method retains the physics-informed loss structure (11) but replaces the neural network ansatz with a compositional search over EML-derived primitives, thereby recovering the symbolic form of the solution.

### 2.4 Symbolic Regression

Symbolic regression seeks to identify a mathematical expression that fits observed data, searching simultaneously over functional forms and numerical coefficients. Two recent systems are particularly relevant.

**PySR** Cranmer (2023) combines genetic programming with local gradient-based refinement. An evolving population of expression trees is subject to mutation, crossover, and selection operators, with periodic gradient descent applied to the numerical constants in each tree. PySR has demonstrated strong performance on a range of scientific benchmarks.

**AI Feynman** Udrescu and Tegmark (2020) takes a complementary approach, exploiting physics-inspired strategies such as dimensional analysis, symmetry detection, and separability testing to decompose complex expressions into simpler sub-problems before applying brute-force or neural-network-guided search.

Both methods excel at recovering algebraic laws of moderate complexity—the paradigmatic examples being $F = ma$ and $E = mc^2$. However, they face significant challenges in the regime targeted by the present work:

- *Multi-variable PDE solutions.* Solutions to PDEs in two or more spatial dimensions typically involve products of univariate functions along different axes (e.g., $\sin(x)\cos(y)$). Standard symbolic regression treats the entire expression as a single tree, making it difficult to exploit this separable structure.
- *Combinatorial explosion.* The space of nested function compositions grows super-exponentially with expression depth. Genetic operators struggle to navigate this space efficiently when the target involves three or more levels of composition.
- *Mixed exponential–trigonometric forms.* Time-dependent solutions with spatial oscillation and temporal decay—such as $\sin(x)\exp(-\alpha t)$—require the simultaneous discovery of exponential and trigonometric primitives with coupled coefficients, a combination that lies in a particularly sparse region of typical expression-tree search spaces.

Our approach addresses these difficulties by constraining the search to a physically motivated basis of EML-derived primitives organized into separable, axis-aligned terms. Rather than evolving arbitrary expression trees, we optimize a structured ansatz whose functional building blocks—$\sin$, $\cos$, $\exp$, $\ln$—are fixed and verified, while only the linear combination coefficients and frequency parameters remain trainable. This reduces the effective search space by orders of magnitude while retaining the capacity to express the product-of-functions structure characteristic of classical PDE solutions.

## 3 Method

We introduce the ComposeHead, a trainable module that represents candidate solutions as sums of primitive compositions, together with a search-and-refine pipeline that converts learned parameters into exact symbolic expressions. The key design principle is to embed the separable product structure common to analytical PDE solutions directly into the architecture, so that the optimizer need only recover scalar coefficients rather than discover structural patterns from scratch.

### 3.1 ComposeHead Architecture

A ComposeHead with $n$ input dimensions represents a function

$$f(\mathbf{x}) \;=\; \beta \;+\; \sum_{i=1}^{T} c_i \, \phi_i(\mathbf{x}), \tag{12}$$

where $\beta \in \mathbb{R}$ is a trainable bias, $c_i \in \mathbb{R}$ are trainable coefficients, and each $\phi_i$ is a *term*—a composition of primitives drawn from a function basis $\mathcal{F} = \{\sin, \cos, \exp, \ln, \mathrm{id}, \mathrm{sq}, \mathrm{inv}\}$. All primitives are implemented via verified EML constructions (Section 2), ensuring that the entire forward pass traces back to the base operation $\mathrm{eml}(x, y) = e^x - \ln y$.

We define three term types of increasing structural commitment.

Table 1: Parameter counts per term type for $n$ input dimensions. The SeparableTerm with $K$ factors is independent of $n$, yielding a favorable scaling for high-dimensional separable targets.

| Term type | Parameters | Scaling |
|---|---|---|
| PrimitiveTerm | $n+2$ | $\mathcal{O}(n)$ |
| ProductTerm | $2n+3$ | $\mathcal{O}(n)$ |
| SeparableTerm ($K$ factors) | $2K+1$ | $\mathcal{O}(K)$, independent of $n$ |

**PrimitiveTerm.** A single primitive applied to a learned linear combination of inputs:

$$\phi(\mathbf{x}) = g\big(\mathbf{w}^\top \mathbf{x} + b\big)\,, \quad g \in \mathcal{F}, \tag{13}$$

with trainable parameters $\mathbf{w} \in \mathbb{R}^n$, $b \in \mathbb{R}$, and coefficient $c \in \mathbb{R}$. The weight vector $\mathbf{w}$ allows the term to attend to arbitrary linear combinations of input dimensions, providing maximum flexibility at the cost of a larger parameter space.

**ProductTerm.** A pairwise product of two PrimitiveTerms sharing a single coefficient:

$$\phi(\mathbf{x}) = g_1\big(\mathbf{w}_1^\top \mathbf{x} + b_1\big) \cdot g_2\big(\mathbf{w}_2^\top \mathbf{x} + b_2\big)\,, \quad g_1, g_2 \in \mathcal{F}. \tag{14}$$

Each factor maintains its own weight vector, enabling expressions such as $\sin(\mathbf{w}_1^\top \mathbf{x})\cos(\mathbf{w}_2^\top \mathbf{x})$. In practice, the optimizer must learn near–one-hot patterns in $\mathbf{w}_1$ and $\mathbf{w}_2$ to isolate individual input dimensions—a requirement that introduces spurious local minima when $n$ is even moderately large.

**SeparableTerm (key innovation).** A product of $K$ axis-aligned factors, each operating on exactly one input dimension:

$$\phi(\mathbf{x}) = \prod_{k=1}^{K} g_k(a_k\, x_{d_k} + b_k)\,, \quad g_k \in \mathcal{F},\;\; d_k \in \{1, \ldots, n\}, \tag{15}$$

where each *AxisTerm* factor has only two trainable parameters: a scale $a_k \in \mathbb{R}$ and a bias $b_k \in \mathbb{R}$. The entire SeparableTerm shares a single coefficient $c \in \mathbb{R}$.

The central observation motivating the SeparableTerm is that the vast majority of known analytical PDE solutions are *separable*—expressible as products of univariate functions of individual coordinates. Classical examples include normal modes of the wave equation, Fourier–series solutions to the heat equation, and the Taylor–Green vortex solution to the Navier–Stokes equations. By making this product-of-univariates structure explicit in the architecture, the optimizer is relieved of the burden of discovering one-hot attention patterns in free weight vectors. It need only recover the scalar scales and biases within each factor, a dramatically simpler optimization landscape.

Table 1 summarizes the parameter counts. The SeparableTerm is notable in that its parameter count depends only on the number of factors $K$ and is *independent* of the input dimensionality $n$.

### 3.2 Training Pipeline

Given data $(\mathbf{X}, \mathbf{y})$ with $\mathbf{X} \in \mathbb{R}^{N \times n}$, we seek a symbolic expression $\hat{f}$ that approximates the mapping $\mathbf{x} \mapsto y$. Algorithm 1 presents the full discovery pipeline.

For concreteness, consider the Taylor–Green vortex with inputs $\mathbf{x} = (x, y, t)$ and basis $\mathcal{F} = \{\sin, \cos, \exp\}$. The candidate enumeration in Step 1 generates $|\mathcal{F}|^3 = 27$ triple SeparableTerms (one per function assignment to the three input dimensions). Each candidate is trained for $N_{\text{search}} = 2{,}000$ steps (Step 2), the best is selected (Step 3), and fine-tuned for $N_{\text{fine}} = 5{,}000$ steps (Step 4). The entire search completes in under two minutes on a single CPU, as each candidate contains only $2K + 1 = 7$ trainable parameters.

**Algorithm 1** Compositional EML Discovery

**Require:** Data $(\mathbf{X}, \mathbf{y})$, input dimension $n$, function basis $\mathcal{F} = \{\sin, \cos, \exp, \ldots\}$
**Ensure:** Symbolic expression $\hat{f}$

1: **Candidate enumeration.** For each assignment of $K$ functions from $\mathcal{F}$ to $K$ input dimensions $(d_1, \ldots, d_K)$, instantiate a SeparableTerm $\phi = \prod_{k=1}^{K} g_k(a_k\, x_{d_k} + b_k)$. $\{|\mathcal{F}|^K$ candidates for fixed dims$\}$

2: **Parallel evaluation.** Train each candidate independently for $N_{\text{search}}$ steps using the Adam optimizer (Paszke et al., 2019), minimizing $\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\big(\hat{y}_i - y_i\big)^2$. Rank candidates by final MSE.

3: **Selection.** Retain the top-1 candidate (or top-$M$ for multi-term targets).

4: **Fine-tuning.** Train the selected term(s) for $N_{\text{fine}}$ additional steps at a reduced learning rate.

5: **Normalization.** Canonicalize the learned parameterization:

- *Exponential factors:* absorb the bias into the coefficient via $c \cdot \exp(ax + b) \;\to\; (c\, e^b) \cdot \exp(ax)$, then set $b \leftarrow 0$.
- *Trigonometric factors:* reduce the phase $b$ to $[-\pi, \pi]$; absorb sign flips using $\sin(x + \pi) = -\sin(x)$ and $\cos(x + \pi) = -\cos(x)$.
- *Negative coefficients:* if $c < 0$ and a $\sin$ factor is present, exploit $\sin(-x) = -\sin(x)$ to make $c$ positive by negating the scale and bias of that factor.

6: **Snapping (optional).** Round each parameter to the nearest element of a target set $\mathcal{S} = \{0, \pm 1, \pm\frac{1}{2}, \pm\frac{1}{3}, \pm\frac{\pi}{2}, \pm\pi, \pm e, \pm\sqrt{2}, \pm\ln 2, \ldots\}$ within tolerance $\tau$. If validation data are available, revert any snap that causes the loss to exceed a maximum ratio $r_{\max}$ relative to the pre-snap loss.

7: **return** Symbolic expression extracted via SymPy conversion of the snapped parameters.

The normalization step (Step 5) is critical for reliable snapping. The parameterization $c \cdot g(ax + b)$ admits gauge freedoms: for exponential factors, the coefficient $c$ and the term $e^b$ can trade off arbitrarily without changing the function value, creating a ridge in the loss landscape along which the optimizer may drift. Normalization breaks these symmetries, concentrating the "meaning" of each parameter and ensuring that the subsequent snapping step compares like with like.

### 3.3 Why This Works

We identify five properties of the proposed approach that collectively explain its effectiveness on the class of problems we consider. The first two—normalization and single-term search—are the core technical contributions; the remaining three provide enabling context.

**Normalization resolves gauge freedom (key contribution).** The parameterization $c \cdot g(ax + b)$ introduces redundancies. For $g = \exp$, the identity $c \cdot \exp(ax + b) = (c\, e^b) \cdot \exp(ax)$ means that $c$ and $b$ are not independently identifiable: the optimizer can distribute scale information between $c$ and $e^b$ arbitrarily, creating a ridge in the loss landscape along which it may drift. For $g = \sin$ or $g = \cos$, the periodicity and parity identities create analogous degeneracies—$\sin(x - \pi) = -\sin(x)$ and $\sin(x - \pi/2) = -\cos(x)$, so a $\sin$ factor with a phase offset of $-\pi/2$ is indistinguishable from a $\cos$ factor with a negated coefficient. The normalization step (Algorithm 1, Step 5) systematically eliminates these gauge freedoms, yielding a canonical form in which each parameter has a unique interpretation. This is the most impactful design choice: without normalization, snapping errors are $0.005$–$0.02$; with it, errors drop to $7 \times 10^{-8}$—a five-order-of-magnitude improvement (Section 4.4).

**Single-term search avoids signal splitting (key contribution).** When a multi-term model is trained end-to-end, the optimizer may distribute the target signal across many terms, each capturing a small fraction of the variance. The resulting coefficient magnitudes are small and difficult to distinguish from noise, making pruning unreliable. By training each candidate SeparableTerm in isolation (Step 2 of Algorithm 1), we force the optimizer to concentrate all model capacity in a single term. The correct candidate achieves near-zero loss; incorrect candidates plateau at high loss. This binary signal—correct versus incorrect—is far easier to threshold than the continuous coefficient magnitudes in a multi-term model.

**Separable ansatz matches target structure.** The solutions to the PDEs we target—heat equation, wave equation, Navier–Stokes in the Taylor–Green regime—are products of univariate functions of the spatial and temporal coordinates. The SeparableTerm directly encodes this product-of-univariates structure, so that a correct solution lies at a point in parameter space rather than in a complex submanifold of a higher-dimensional space.

**EML provides a principled basis.** The choice of primitive library $\mathcal{F} = \{\sin, \cos, \exp, \ln, \mathrm{id}, \mathrm{sq}, \mathrm{inv}\}$ is not *ad hoc*. Every element of $\mathcal{F}$ admits a verified construction from the universal EML function $\mathrm{eml}(x, y) = e^x - \ln y$ (Section 2.2). The universality theorem of Odrzywołek (2026) guarantees that this basis can, in principle, generate any computable function through further composition. More practically, the EML function class is closed under partial differentiation (Appendix A), which ensures that PDE residual computations—requiring $\partial_t u$, $\partial_x u$, $\partial_{xx} u$, etc.—remain within the same function class. This closure property is what enables physics-informed training through the compositional graph.

**Gradient-based optimization.** Every operation in our pipeline is differentiable. Each EML primitive—$\exp$ via $\mathrm{eml}(x, 1)$, $\sin$ and $\cos$ via the imaginary and real parts of $\mathrm{eml}(ix, 1) = e^{ix}$—supports standard backpropagation. This allows us to leverage mature gradient-based optimizers (Adam) with well-understood convergence properties, rather than relying on stochastic search heuristics. The per-candidate optimization problem is low-dimensional ($2K + 1$ parameters), smooth, and—for the correct function assignment—typically convex in the neighborhood of the true solution.

## 4 Experiments

We evaluate the proposed pipeline on six classes of problems: separable PDE benchmarks spanning one- and two-dimensional domains (Section 4.1), a non-separable traveling-wave recovery that demonstrates multi-term search (Section 4.2), robustness and failure-mode analysis (Section 4.3), ablation studies isolating each design choice (Section 4.4), PDE-residual-only discovery that recovers exact solutions from the governing equations alone without any solution data (Section 4.5), and a systematic three-dimensional Navier–Stokes screening that tests which classes of 3-D initial conditions admit exact separable solutions (Section 4.6). Section 4.7 collects all results in a unified comparison table. All experiments use PyTorch 2.0+ (Paszke et al., 2019) on a single CPU; the most expensive search (81 two-term pairs for the traveling wave) completes in under ten minutes.

### 4.1 Separable PDE Benchmarks

We begin with three single-term separable PDEs whose exact solutions are products of axis-aligned elementary functions. In each case the pipeline recovers the ground truth to within machine precision after snapping.

#### 4.1.1 1-D Heat Equation

The one-dimensional heat equation $u_t = \nu\, u_{xx}$ on $x \in [0, 1]$, $t \in [0, 1]$ with $\nu = 1/\pi^2$ admits the exact solution

$$u(x, t) = \sin(\pi x)\, \exp(-\pi^2 \nu\, t) = \sin(\pi x)\, \exp(-t). \tag{16}$$

**Setup.** We sample $n = 2{,}000$ training points uniformly over the domain and set the function basis to $\mathcal{F} = \{\sin, \cos, \exp\}$, yielding $|\mathcal{F}|^2 = 9$ two-factor separable candidates of the form $f(x)\,g(t)$. Each candidate is trained for 2,000 steps with Adam (learning rate $0.01$, gradient clipping at max-norm $1.0$); the best candidate is then fine-tuned for 5,000 steps at learning rate $0.005$.

**Result.** The search selects $\sin(\cdot)\cdot\exp(\cdot)$ as the lowest-loss candidate (loss $< 10^{-9}$ after fine-tuning). After normalization and snapping, the recovered expression is $\hat{u}(x,t) = \sin(\pi x)\,\exp(-t)$, with snap MAE of $9 \times 10^{-8}$ on 5,000 held-out points.

#### 4.1.2 1-D Wave Equation

The one-dimensional wave equation $u_{tt} = c^2\,u_{xx}$ on $x \in [0,1]$, $t \in [0,2]$ with $c = 1$ and initial conditions $u(x,0) = \sin(\pi x)$, $u_t(x,0) = 0$ has the standing-wave solution

$$u(x,t) = \sin(\pi x)\,\cos(\pi t). \tag{17}$$

**Setup.** The configuration is identical to the heat equation: $n = 2{,}000$ training points, basis $\mathcal{F} = \{\sin, \cos, \exp\}$, 9 candidates, 2,000 search steps followed by 5,000 fine-tuning steps.

**Result.** The winning candidate is $\sin(\cdot)\cdot\cos(\cdot)$ with loss $< 10^{-9}$. Snapping yields $\hat{u}(x,t) = \sin(\pi x)\,\cos(\pi t)$ with snap MAE of $5 \times 10^{-8}$.

#### 4.1.3 2-D Navier–Stokes: Taylor–Green Vortex

The Taylor–Green vortex is an exact, time-decaying solution to the two-dimensional incompressible Navier–Stokes equations (Taylor and Green, 1937):

$$u(x,y,t) = \quad \sin(x)\cos(y)\,\exp(-2\nu t), \tag{18}$$

$$v(x,y,t) = -\cos(x)\sin(y)\,\exp(-2\nu t), \tag{19}$$

$$p(x,y,t) = -\tfrac{1}{4}\big[\cos(2x) + \cos(2y)\big]\exp(-4\nu t), \tag{20}$$

where $\nu = 0.1$ is the kinematic viscosity. Because the velocity field is a product of separable, axis-aligned elementary functions, this problem provides an ideal test of whether the COMPOSEHEAD can recover exact symbolic structure from data alone.

**Setup.** We sample $n = 3{,}000$ training points uniformly over $x, y \in [0, 2\pi]$, $t \in [0,1]$ and evaluate both velocity components at the exact solution (18)–(19). The function basis is $\mathcal{F} = \{\sin, \cos, \exp\}$, yielding $|\mathcal{F}|^3 = 27$ triple separable candidates of the form $f(x)\,g(y)\,h(t)$. Each candidate is trained independently for 2,000 steps with Adam (learning rate $0.01$, gradient clipping at max-norm $1.0$). The best candidate for each velocity component is then fine-tuned for an additional 5,000 steps at learning rate $0.005$.

**Search results.** For the $u$-component, the search identifies $\sin(\cdot)\cdot\sin(\cdot)\cdot\exp(\cdot)$ as the lowest-loss candidate (loss $< 10^{-8}$); for $v$, the winner is $\cos(\cdot)\cdot\cos(\cdot)\cdot\exp(\cdot)$. These selections appear to differ from the ground truth, which contains $\sin{\cdot}\cos$ and $\cos{\cdot}\sin$ factors, respectively. The discrepancy is resolved by observing that the optimizer absorbs phase shifts into the bias parameters: $\sin(x-\pi)\sin(y-\pi/2)\exp(at+b) = \sin(x)\cos(y)\exp(at)\exp(b)$ up to sign, so the learned representation is equivalent to the true solution modulo a reparameterization that the normalization step removes.

Table 2: Comparison of methods on the Taylor–Green vortex. "Data Error" is the mean squared error on the training data after convergence. "Symbolic Output" indicates whether the method produces a human-readable formula.

| Method | Data Error | Symbolic Output | Interpretable |
|---|---|---|---|
| MLP-PINN (3-layer, 64-wide) | $\sim 10^{-3}$ | No (weight matrix) | No |
| Raw EML tree (depth 4) | $\sim 0.28$ | Opaque nested `eml()` | Barely |
| COMPOSEHEAD (free weights) | $\sim 10^{-3}$ | Messy multi-term sum | Partially |
| COMPOSEHEAD (separable) | $7\times10^{-8}$ | $\sin(x)\cos(y)\exp(-t/5)$ | **Yes** |

**Raw learned parameters.** After fine-tuning, the combined MSE loss reaches $\sim 10^{-10}$. The raw parameters before normalization are as follows:

- $u$: coeff $= 0.540$; $\sin_0$: $a = 1.000$, $b = -3.142$ $(\approx -\pi)$; $\sin_1$: $a = 1.000$, $b = -1.571$ $(\approx -\pi/2)$; $\exp_2$: $a = -0.200$, $b = 0.616$.
- $v$: coeff $= 0.594$; $\cos_0$: $a = 1.000$, $b = 0.000$; $\cos_1$: $a = 1.000$, $b = 1.571$ $(\approx \pi/2)$; $\exp_2$: $a = -0.200$, $b = 0.522$.

**Normalization and snapping.** The normalization step (i) absorbs exponential biases into the coefficient ($e^{0.616} \approx 1.851$, yielding $0.540 \times 1.851 \approx 1.000$), (ii) canonicalizes trigonometric phases by applying the identities $\sin(x-\pi) = -\sin(x)$ and $\sin(y-\pi/2) = \cos(y-\pi) \mapsto -\cos(y)$ together with the sign-absorption rule for $\sin$, and (iii) reduces all bias terms to zero. After normalization, coefficient snapping with tolerance $0.05$ yields the exact closed-form expressions:

$$\hat{u} = \sin(x)\cos(y)\exp(-t/5), \qquad \hat{v} = -\cos(x)\sin(y)\exp(-t/5). \tag{21}$$

The snap error—defined as the mean absolute difference between the snapped model and the ground truth on 5,000 held-out points—is $7 \times 10^{-8}$, consistent with machine precision.

**PDE residual verification.** We verify the recovered solution against the governing equations. The continuity residual $|\nabla \cdot \mathbf{u}|$ has mean value $3 \times 10^{-4}$, attributable to finite-difference approximation of the divergence. The momentum residuals are non-zero because the pipeline does not model the pressure field; the residual in each momentum equation equals $\nabla p$, which is consistent with the pressure gradient implied by (20).

**Comparison with baselines.** Table 2 compares four approaches on the same Taylor–Green data set.

The MLP-PINN (Raissi et al., 2019) achieves low data error but yields an opaque weight matrix. The raw EML tree (depth 4) struggles with the three-variable product structure and converges to a poor local minimum. The COMPOSEHEAD with free (non-axis-aligned) weight vectors fits the data reasonably but distributes signal across multiple terms, producing a multi-term expression that resists simplification. Only the separable COMPOSEHEAD recovers the exact symbolic solution.

### 4.2 Non-Separable Recovery via Multi-Term Search

The single-term separable ansatz $f(x)\,g(t)$ cannot represent functions whose natural decomposition requires a sum of separable products. A canonical example is the traveling wave

$$u(x,t) = \sin(x - 2t), \tag{22}$$

which by the angle-subtraction identity decomposes as

$$u(x,t) = \sin(x)\cos(2t) - \cos(x)\sin(2t). \tag{23}$$

Table 3: Noise robustness on the Taylor–Green $u$-component. For $\sigma \leq 0.1$ the method recovers the exact symbolic expression; at $\sigma = 0.2$ recovery fails.

| Noise level $\sigma$ | Snap MAE | Exact recovery? |
|---|---|---|
| 0.0 | $7\times10^{-8}$ | Yes |
| 0.001 | $8\times10^{-8}$ | Yes |
| 0.01 | $6\times10^{-8}$ | Yes |
| 0.05 | $4\times10^{-7}$ | Yes |
| 0.1 | $9\times10^{-7}$ | Yes |
| 0.2 | — | No |

This two-term structure lies strictly outside the single-product function class but within the class of sums of separable products. Recovering it tests whether the pipeline can extend beyond the single-term setting.

**Setup.** We sample $n = 2{,}000$ training points uniformly over $x \in [0, 2\pi]$, $t \in [0, 2]$. The function basis is $\mathcal{F} = \{\sin, \cos, \exp\}$, giving $|\mathcal{F}|^2 = 9$ possible single-term candidates per factor. For the two-term search, we enumerate all $9 \times 9 = 81$ ordered pairs of function assignments, where each pair consists of two separable products $c_1\, f_1(x)\, g_1(t) + c_2\, f_2(x)\, g_2(t)$. Each pair is trained jointly for 2,000 steps with Adam (learning rate $0.01$), and the best pair is fine-tuned for 5,000 additional steps at learning rate $0.005$.

**Bias freezing.** A critical implementation detail is that *all bias parameters are frozen at zero* during the multi-term search. The reason is a gauge freedom inherent to multi-term decompositions: if the two terms sum to a fixed target, then correlated phase offsets—such as adding $\delta$ to the bias of $f_1$ and simultaneously adjusting $c_1$, $c_2$, and the bias of $f_2$ to compensate—leave the total sum unchanged while preventing any individual term from snapping to a clean symbolic form. In the single-term setting this gauge freedom is absent because the normalization step can absorb all phase offsets into the coefficient and canonicalize each factor independently. Freezing biases at zero eliminates the multi-term gauge manifold and forces the optimizer to express the target using canonical basis functions directly.

**Result.** The single-term search correctly identifies that no single product achieves low loss: all 9 candidates plateau at loss $> 0.13$ (see Section 4.3). In the two-term search, the winning pair is $\sin(x)\cos(t) + \cos(x)\sin(t)$, achieving loss $< 10^{-8}$ after fine-tuning. After coefficient snapping, the recovered expression is

$$\hat{u}(x, t) = \sin(x)\cos(2t) - \cos(x)\sin(2t), \tag{24}$$

with snap MAE of $3 \times 10^{-8}$ on 5,000 held-out points—again consistent with machine precision. This confirms that the separable ansatz extends naturally to *sums* of separable terms, covering a strictly larger function class than single-product solutions.

### 4.3 Robustness and Failure Analysis

A trustworthy symbolic-recovery method should not only succeed on clean data but also degrade gracefully under adverse conditions. We examine three scenarios: observation noise, a misspecified function basis, and a single-term search applied to a non-separable target.

#### 4.3.1 Noise Robustness

We add i.i.d. Gaussian noise $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ to the Taylor–Green training data and repeat the full pipeline (27-candidate search, fine-tuning, normalization, snapping) for each noise level. Table 3 reports the snap MAE of the recovered $u$-component.

Table 4: Ablation study on the Taylor–Green vortex. Each row modifies one component of the default pipeline (separable single-term search with normalization). "Snap Error" is the mean absolute error of the snapped model on 5,000 held-out points.

| Variant | Snap Error | Outcome |
|---|---|---|
| **Full pipeline (default)** | $7\times10^{-8}$ | Exact recovery |
| Free-weight terms (non-separable) | $\sim10^{-3}$ | Multi-term local minimum |
| Multi-term with $L_1$ regularization | $\sim10^{-2}$ | Over-pruning of 60+ competing terms |
| Without normalization (direct snap) | 0.005–0.02 | Phase offsets corrupt snapping |

At noise levels $\sigma \leq 0.1$ (corresponding to a signal-to-noise ratio of roughly 10 or greater, given that the Taylor–Green velocity field has unit amplitude), the correct candidate is selected and the snapped parameters match the ground truth. The snap MAE increases modestly with $\sigma$ but remains below $10^{-6}$—well within the snapping tolerance. At $\sigma = 0.2$ the noise overwhelms the signal, the search selects an incorrect candidate, and snapping fails. The method thus tolerates moderate observation noise, a property it inherits from the continuous relaxation: the gradient-based optimizer is a natural denoiser that averages over the training set during each update.

#### 4.3.2 Wrong Basis

We repeat the Taylor–Green search with a deliberately misspecified basis $\mathcal{F}' = \{\exp, \ln\}$, omitting all trigonometric functions. This yields $|\mathcal{F}'|^3 = 8$ candidates. Every candidate plateaus at training loss $> 0.18$, far above the $< 10^{-8}$ threshold achieved by the correct basis. Importantly, the method does not produce a spurious "recovery": the large residual loss signals unambiguously that no candidate in the pool is adequate. This behavior is desirable—a symbolic regression system should fail loudly when the hypothesis class does not contain the target, rather than returning a misleading approximate fit (Cranmer, 2023).

#### 4.3.3 Non-Separable Target with Single-Term Search

Applying the single-term separable search to the traveling wave $u(x,t) = \sin(x - 2t)$ provides a complementary failure mode. All 9 candidates of the form $f(x)\,g(t)$ converge to training losses above $0.13$, correctly indicating that no single separable product can represent the target. This gap between the single-term plateau ($> 0.13$) and the two-term optimum ($< 10^{-8}$) provides a reliable diagnostic: when the best single-term loss is orders of magnitude above machine precision, the user is directed to attempt a multi-term search (Section 4.2).

### 4.4 Ablation Study

We isolate the effect of each design decision on the Taylor–Green benchmark. Table 4 summarizes the results.

**Separable vs. free-weight terms.** When each factor's weight vector is a free $\mathbb{R}^3$ vector rather than an axis-aligned scalar, the optimizer converges to a representation that spreads signal across multiple terms. The resulting multi-term sum achieves reasonable data error ($\sim 10^{-3}$) but does not simplify to the known single-product expression, because the weight sharing across input dimensions introduces a degenerate manifold of equivalent parameterizations.

**Single-term search vs. multi-term with $L_1$.** A natural alternative to our candidate-by-candidate search is to instantiate all 27 separable terms simultaneously and apply $L_1$ regularization to the coefficients to encourage sparsity (Brunton et al., 2016). In practice, this approach over-prunes: with 60+ competing terms (27 for each velocity component plus cross-terms), the $L_1$ penalty suppresses the correct term before the optimizer can distinguish it from its neighbors. Single-term search avoids this failure mode by giving each candidate the full optimization budget in isolation.

**Effect of normalization.** Without the normalization step, snapping must operate directly on the raw learned parameters, which carry arbitrary phase offsets ($b \approx -\pi, -\pi/2$) and exponential biases ($b \approx 0.6$). Rounding these values to the nearest "clean" constants introduces errors of $0.005$–$0.02$, two to five orders of magnitude worse than snapping after normalization. The normalization step is therefore essential for exact recovery.

**Candidate pool coverage.** Among the 27 candidates searched for each velocity component, the top 4 candidates all achieve final loss below $10^{-7}$. These correspond to the four function triples that are related to the ground truth by the $\sin \leftrightarrow \cos$ phase-shift equivalence (e.g., $\sin{\cdot}\sin{\cdot}\exp$ and $\cos{\cdot}\cos{\cdot}\exp$ for the $u$-component both encode $\sin(x)\cos(y)\exp(-t/5)$ under suitable phase offsets). The remaining 23 candidates converge to losses above $10^{-2}$, providing a clear separation that makes candidate selection unambiguous.

### 4.5 PDE-Residual-Only Discovery

All preceding experiments supply solution data—values of the target function at sampled points—and use standard regression loss to drive training. A more ambitious question is whether the pipeline can discover exact analytical solutions from the governing equations alone, without *any* solution data. This is the setting of physics-informed neural networks (PINNs) (Raissi et al., 2019), but with a critical difference: the output is a closed-form symbolic expression rather than a set of neural-network weights.

**Formulation.** We consider the two-dimensional incompressible Navier–Stokes equations in vorticity form, which eliminates the pressure variable:

$$\omega_t + u\,\omega_x + v\,\omega_y = \nu\,(\omega_{xx} + \omega_{yy}), \qquad u_x + v_y = 0, \qquad \omega = v_x - u_y, \tag{25}$$

where $\nu = 0.1$. The loss function contains no solution data; it consists entirely of PDE residuals evaluated at randomly sampled collocation points plus an initial-condition penalty:

$$\mathcal{L} = \mathcal{L}_{\mathrm{PDE}} + \lambda\,\mathcal{L}_{\mathrm{IC}}, \tag{26}$$

where $\mathcal{L}_{\mathrm{PDE}}$ is the mean-squared vorticity-equation and continuity residuals computed via automatic differentiation, and $\mathcal{L}_{\mathrm{IC}}$ penalizes deviation from the Taylor–Green initial conditions $u_0 = \sin(x)\cos(y)$, $v_0 = -\cos(x)\sin(y)$ at $t = 0$.

**Setup.** The search space consists of 27 separable candidates for $u$ (basis $\mathcal{F} = \{\sin, \cos, \exp\}$, three factors for $x$, $y$, $t$). For each $u$-candidate, the $v$-candidate is constructed by swapping $\sin \leftrightarrow \cos$ in the $x$ and $y$ factors, motivated by the continuity constraint. Phase 1 trains each $(u, v)$ pair for 1,000 steps on the initial-condition loss plus a continuity penalty ($n_{\mathrm{coll}} = 1{,}500$, $n_{\mathrm{IC}} = 500$). Phase 2 fine-tunes the best pair for 3,000 steps with the full vorticity-equation residual ($\lambda = 10$). Phase 3 applies normalization and snapping.

**Result.** Phase 1 identifies $u \sim \sin(x)\cdot\cos(y)\cdot\exp(t)$, $v \sim \cos(x)\cdot\sin(y)\cdot\exp(t)$ as the lowest-loss pair. During Phase 2, the PDE residual drops to $\sim 10^{-8}$, with the vorticity and continuity components both converging to effectively zero. After normalization and snapping, the recovered solution is

$$\hat{u} = \exp(-t/5)\sin(x)\cos(y), \qquad \hat{v} = -\exp(-t/5)\sin(y)\cos(x), \tag{27}$$

matching the known Taylor–Green solution (18)–(19) with MAE of $5 \times 10^{-8}$ for both components.

**Significance.** This result demonstrates that the COMPOSEHEAD can function as a *symbolic PINN*—a physics-informed solver whose output is not an opaque weight matrix but a human-readable, analytically verifiable closed-form expression. The standard PINN paradigm produces a numerical approximation that can be evaluated at arbitrary points

but whose functional form is unknown. Here, the discovered expression $\sin(x)\cos(y)\exp(-t/5)$ can be substituted back into the Navier–Stokes equations *analytically* and verified to be an exact solution—a qualitatively different kind of result that no conventional neural PDE solver can provide.

### 4.6 Three-Dimensional Navier–Stokes: Gradient-Condition Screening

The preceding experiments operate in two spatial dimensions. A natural question is whether the pipeline extends to three-dimensional Navier–Stokes, where the vortex stretching term $(\boldsymbol{\omega}\cdot\nabla)\mathbf{u}$—absent in 2-D—introduces fundamentally richer nonlinear dynamics.

**Mathematical framework.** For a separable velocity field $\mathbf{u} = \mathbf{u}_0(x, y, z)\, e^{-\nu k^2 t}$ to satisfy the 3-D incompressible Navier–Stokes equations exactly, three conditions must hold: (i) $\nabla\cdot\mathbf{u}_0 = 0$, (ii) $\nabla^2\mathbf{u}_0 = -k^2\mathbf{u}_0$, and (iii) the *gradient condition*

$$(\mathbf{u}_0\cdot\nabla)\boldsymbol{\omega}_0 = (\boldsymbol{\omega}_0\cdot\nabla)\mathbf{u}_0, \tag{28}$$

where $\boldsymbol{\omega}_0 = \nabla\times\mathbf{u}_0$. When this holds, the nonlinear term is a pure gradient and is absorbed entirely into the pressure. We evaluate condition (28) computationally at $5{,}000$ random spatial points for each candidate IC.

**Systematic screening.** We test 13 divergence-free initial conditions spanning four families:

- **Family A** (8 ICs): single-product fields of the form $(A\sin x\cos y\cos z,\ B\cos x\sin y\cos z,\ C\cos x\cos y\sin z)$ with $A + B + C = 0$, wavenumber $k^2 = 3$.
- **Family B** (2 ICs): higher spatial frequency $(\sin 2x,\ \cos 2x,\ \cos 2x)$ variants, $k^2 = 6$.
- **Family C** (1 IC): mixed wavenumber $(\sin x\cos y\cos 2z, \ldots)$, $k^2 = 6$.
- **Family D** (2 ICs): uniform wavenumber-2 fields, $k^2 = 12$.

Every one of the 13 ICs fails the gradient condition with $\max|\mathbf{S}| \geq 0.7$, confirming that no single-product three-dimensional initial condition in these families admits an exact exponentially-decaying solution. The screening runs in under five seconds on a single CPU.

**Why single-product 3-D solutions are rare.** The gradient condition (28) is automatically satisfied for Beltrami flows ($\boldsymbol{\omega} = \lambda\mathbf{u}$), because both sides reduce to $\lambda(\mathbf{u}\cdot\nabla)\mathbf{u}$. For non-Beltrami fields, the vortex stretching term $(\boldsymbol{\omega}\cdot\nabla)\mathbf{u}$ generically differs from the advection term, and the nonlinear residual cannot be absorbed into a scalar pressure gradient. This provides a computational explanation for the well-known scarcity of exact 3-D Navier–Stokes solutions.

**Beltrami verification.** As a positive control, we apply the PDE-residual-only solver to Arnold–Beltrami–Childress (ABC) flows $\mathbf{u}_0 = (A\sin z + C\cos y,\ B\sin x + A\cos z,\ C\sin y + B\cos x)$, which satisfy $\boldsymbol{\omega}_0 = \mathbf{u}_0$ for all $A, B, C$. The solver recovers $\mathbf{u} = \mathbf{u}_0\, e^{-\nu t}$ at MAE $< 10^{-8}$ for both the symmetric case $(A, B, C) = (1, 1, 1)$ and the asymmetric case $(A, B, C) = (1, \sqrt{2}, \sqrt{3})$, confirming that the pipeline handles multi-term 3-D solutions when they exist.

**Multi-term search for non-Beltrami solutions.** We also test a four-term-per-component ansatz on the non-Beltrami IC $(1, 1, -2)$, allowing the optimizer to discover additional spatial modes generated by nonlinear interactions. The vorticity-equation residual plateaus at $\sim 0.4$ after $3{,}000$ training steps, indicating that the nonlinear mode coupling does not close within a finite set of four separable products. This is consistent with the physical picture of 3-D turbulence: vortex stretching cascades energy to ever-finer scales, preventing finite-dimensional mode closure.

Table 5: Comprehensive comparison across all experiments. "Terms" is the number of separable products in the recovered expression. Snap MAE is evaluated on 5,000 held-out points. "Exact?" indicates whether the snapped expression matches the ground truth to within machine precision.

| Example | Type | Terms | Snap MAE | Outcome |
|---|---|---|---|---|
| 1-D Heat equation | Separable | 1 | $9\times10^{-8}$ | Exact recovery |
| 1-D Wave equation | Separable | 1 | $5\times10^{-8}$ | Exact recovery |
| Taylor–Green vortex | Separable | 1 | $7\times10^{-8}$ | Exact recovery |
| Traveling wave $\sin(x-2t)$ | Non-sep. | 2 | $3\times10^{-8}$ | Exact recovery |
| Taylor–Green (PDE-residual only) | No data | 1 | $5\times10^{-8}$ | Exact recovery |
| 3-D ABC Beltrami $(1,\sqrt{2},\sqrt{3})$ | No data | 2 | $1\times10^{-8}$ | Exact recovery |
| 13 non-Beltrami 3-D ICs | — | — | — | Correct failure |
| Wrong basis ($\{\exp,\ln\}$) | — | — | — | Correct failure |
| High noise ($\sigma=0.2$) | — | — | — | Correct failure |

### 4.7 Summary of Results

Table 5 consolidates all experimental outcomes. The pipeline achieves exact symbolic recovery (snap MAE at or below $10^{-7}$) for every target that lies within the separable or sum-of-separable hypothesis class, and produces clear failure signals for targets outside the class or for misspecified basis sets.

Several observations are worth highlighting. First, the snap MAE is remarkably consistent across all successful recoveries, ranging from $3 \times 10^{-8}$ to $9 \times 10^{-8}$—essentially the residual of double-precision arithmetic rather than an approximation error. Second, the multi-term search for the traveling wave achieves the *same* precision as the single-term cases, confirming that bias freezing is an effective substitute for single-term normalization when the decomposition involves more than one product. Third, the two failure modes—wrong basis and excessive noise—produce losses that are separated from the success cases by five or more orders of magnitude, making it straightforward to distinguish genuine recovery from spurious fits without manual inspection.

Compared to general-purpose symbolic regression tools such as PySR (Cranmer, 2023) and AI Feynman (Udrescu and Tegmark, 2020), the separable COMPOSEHEAD offers a narrower hypothesis class but, within that class, achieves machine-precision recovery with a deterministic, gradient-based search that requires no genetic operators, no library of heuristic transformations, and no human-in-the-loop simplification. The key limitation—restriction to sums of axis-aligned separable products—is discussed further in Section 5.

## 5 Discussion

The experiments in Section 4 demonstrate that the COMPOSEHEAD can recover exact closed-form solutions to the two-dimensional Navier–Stokes equations from data. We now examine what makes the pipeline work, situate it relative to existing methods, and identify the limitations that constrain its present applicability.

**What the method can do.** The method discovers closed-form PDE solutions whenever the true solution can be expressed as a linear combination of products of $\sin$, $\cos$, $\exp$, $\ln$, and the identity applied along individual coordinate axes. This class encompasses a substantial fraction of the analytical solutions in classical references: separable solutions to heat, wave, and advection–diffusion equations; Fourier modes of elliptic boundary-value problems; and exponentially decaying vortex solutions of the Navier–Stokes equations. The ansatz (12) aligns naturally with the classical method of separation of variables, a primary tool for constructing exact PDE solutions for over three centuries. The novelty is not the separable form itself but the end-to-end pipeline from data to exact symbolic expressions without requiring human insight into the solution structure. The method also extends to non-separable solutions that decompose as finite sums of separable terms. The traveling wave $\sin(x - 2t) = \sin(x)\cos(2t) - \cos(x)\sin(2t)$ was recovered exactly via

a brute-force two-term pair search, demonstrating that the effective hypothesis class is strictly larger than the set of single separable products.

**What the method cannot do.** The COMPOSEHEAD does not prove existence or regularity of solutions in the PDE-theoretic sense, does not address uniqueness, and does not guarantee that a discovered solution is the physically relevant one. Turbulent flows lie entirely outside the method's reach. More generally, any solution not expressible as a finite linear combination of separable products of the available primitives is undiscoverable.

**Relationship to the Navier–Stokes Millennium Prize.** The recovery of the Taylor–Green vortex naturally raises the question of how this work relates to the Clay Mathematics Institute Millennium Prize Problem. We wish to be precise: the Prize requires proving, for the three-dimensional incompressible Navier–Stokes equations, that smooth solutions exist globally in time for arbitrary smooth initial data of finite energy, or exhibiting a counterexample. Our work addresses a fundamentally different question: given specific initial conditions, can one algorithmically discover a closed-form expression that satisfies the equations? The result is a *constructive existence demonstration* for a particular initial datum—we exhibit the solution rather than proving one must exist. This is distinct from, and considerably weaker than, the general existence claim demanded by the Prize. Nonetheless, algorithmically discovering closed-form solutions for families of initial conditions may inform theoretical investigations by providing exact solutions whose analytic properties can be studied directly.

**The role of normalization.** The normalization pipeline is the central practical contribution of this work. The parameterization $c_k \cdot f(\alpha_{k,d}\, x_d + \beta_{k,d})$ admits gauge freedom: multiple parameter settings encode the same function. For example, $\sin(x-\pi)\cdot\exp(b)\cdot c = \sin(x)\cdot c'$ where $c' = -\exp(b)\cdot c$. A cosine with phase $\pi/2$ is a sine; exponential biases can be absorbed into multiplicative coefficients. Without normalization, gradient-based optimization converges to one of many equivalent settings, and coefficient snapping fails. The normalization pipeline resolves this gauge freedom by absorbing exponential biases into $c_k$, canonicalizing trigonometric phases via $\sin(x+\pi/2) = \cos(x)$ and $\cos(x+\pi) = -\cos(x)$, and reducing all phase offsets to $[0, \pi/2)$. This canonicalization transforms a continuously parameterized model into an expression that coefficient snapping can convert to exact symbolic form, reducing parameter errors from arbitrary gauge-equivalent configurations to the order of $10^{-6}$.

**Single-term search.** Equally important is searching for one term at a time rather than fitting a multi-term linear combination simultaneously. Joint optimization allows the optimizer to split a single physical signal across several basis elements—representing $\sin(x)\cos(y)$ as a sum of two terms with compensating coefficients. Such splitting yields numerically accurate fits but expressions bearing no resemblance to the true solution. By fitting a single term, subtracting it from the residual, and repeating, each term is forced to capture a complete, interpretable component. This greedy strategy is a practical choice rather than a theoretical necessity, but it proves essential for exact, human-readable recovery.

**What EML provides (and what it does not).** The Euler–Maclaurin–Lie framework contributes two things. First, it provides a principled, non-ad-hoc justification for the primitive basis $\{\sin, \cos, \exp, \ln\}$: these arise as fundamental solutions of linear constant-coefficient differential equations and their multiplicative counterparts. Without EML the same library could be chosen on empirical grounds, but the basis selection would lack theoretical grounding. Second, the EML basis is closed under differentiation, guaranteeing that PDE residuals computed from a candidate expression remain within the representable class and that residual-based loss functions stay well-defined throughout optimization.

The normalization pipeline, however, does not follow from EML per se. The gauge-resolving identities exploit specific algebraic properties of $\exp$, $\sin$, and $\cos$ that hold independently of the EML framework. EML determines *which* functions to include; normalization determines *how* to make optimization over those functions yield exact symbolic output.

EML universality motivated the architectural choices that created the conditions under which normalization and snapping could succeed. The conceptual chain—universality $\rightarrow$ verified primitives $\rightarrow$ compositional head $\rightarrow$

Table 6: EML branch closure predictions and computational verification. "Closure" indicates the mechanism by which the nonlinear term stays within the separable hypothesis class. All predictions were confirmed.

| PDE | Nonlinearity | Branch | Closure | Outcome |
|---|---|---|---|---|
| 1-D Heat | Linear | T | Direct | Exact recovery |
| 1-D Wave | Linear | T | Direct | Exact recovery |
| 2-D Navier–Stokes | Quadratic | T | Gradient | Exact recovery |
| 3-D NS (Beltrami) | Quadratic | T | Beltrami | Exact recovery |
| 3-D NS (non-Beltrami) | Quadratic | T | Fails | Correct failure |
| 1-D Burgers | Quadratic | T,E | Fails | Correct failure |
| 1-D KdV | Quadratic | T | Fails | Correct failure |
| NLS standing wave | Cubic | T | Fails | Correct failure |

structured parameters $\to$ normalization $\to$ exact recovery—starts with EML. However, the normalization pipeline itself exploits properties of $\exp$, $\sin$, and $\cos$ that are independent of the EML implementation, and the numerical results would be identical if the primitives were implemented via PyTorch intrinsics rather than EML compositions. EML universality motivates the framework and guarantees completeness of the primitive basis, but the practical contributions—normalization and search strategy—are independent of the EML implementation and would apply equally to any library of elementary functions.

**EML branch closure: predicting solution existence.** The EML-derived primitives naturally partition into three *branches* based on their behavior under differentiation and multiplication: **Branch E** ($\exp$), which is closed under both operations; **Branch T** ($\sin$, $\cos$), which is closed under differentiation but generates new frequencies under multiplication ($\sin a \cdot \sin b \to \cos(a-b) - \cos(a+b)$); and **Branch L** ($\ln$), which is not closed under differentiation ($\mathrm{d}/\mathrm{d}x\ \ln x = 1/x$). A PDE's nonlinear operator $N$ maps solutions from one branch combination to another, and the existence of exact separable solutions depends on whether this mapping *closes*:

- **Direct closure.** If $N$ maps a branch back to itself (as linear operators do for Branch T), exact separable solutions exist generically. The heat and wave equations fall into this category.
- **Gradient closure.** If $N$ generates terms outside the branch but those terms are a pure gradient—absorbable into an auxiliary variable such as pressure—the equation is effectively closed. The 2-D Navier–Stokes equations satisfy this condition: the quadratic term $(\mathbf{u}\cdot\nabla)\mathbf{u}$ produces $\sin(2x)$ from $\sin(x)$, but this equals $\partial_x[-\frac{1}{4}\cos(2x)]$ and is absorbed into $\nabla p$.
- **No closure.** When neither mechanism applies, the nonlinearity generates irreducible new modes, and no finite separable solution exists.

This framework yields verifiable predictions. We tested it on eight PDEs (Table 6): the Burgers, KdV, and nonlinear Schrödinger equations all fail closure in Branch T, which the PDE-residual solver confirms by producing residuals orders of magnitude above machine precision. The 3-D non-Beltrami Navier–Stokes equations fail gradient closure because vortex stretching introduces an irreducible term (Section 4.6). All four predictions of success and all four predictions of failure were confirmed computationally, suggesting that branch closure analysis can serve as a *pre-screening tool* that identifies which PDEs are amenable to separable symbolic recovery before any optimization is attempted.

**Comparison to symbolic regression.** PySR Cranmer (2023) and AI Feynman Udrescu and Tegmark (2020) search over arbitrary expression trees and are strictly more general: they can discover algebraic relationships of essentially any form, including non-separable expressions such as $E = mc^2$. Our approach trades this generality for structure. The trade-off is justified by a concrete gap. General symbolic regression methods struggle with multi-variable separable products common in PDE solutions: recovering $\sin(x)\cos(y)\exp(-t/5)$ requires simultaneously discovering the

correct univariate functions, their arguments, and the multiplicative coupling—a combinatorially explosive task. Physics-informed neural networks avoid this search but produce weight matrices, not symbolic formulas. The COMPOSEHEAD fills this gap by constraining the hypothesis class to separable products, where free parameters scale linearly in the number of terms rather than exponentially in expression depth.

**Limitations.**

- *Data dependence.* The method requires training data from simulation, experiment, or collocation-point sampling; it cannot discover solutions from the PDE alone.
- *Separability assumption.* The ansatz (12) cannot represent non-separable solutions such as self-similar forms $u = t^{-\alpha} g(x/t^{\beta})$ or traveling waves $u = f(x - ct)$.
- *Combinatorial scaling.* The candidate count scales as $|\mathcal{F}|^K \times D^K$, exponential in the product order $K$.
- *Dimensionality.* Experiments are restricted to two spatial dimensions plus time; three-dimensional validation remains future work.
- *Basis completeness.* The primitive library must contain the functions in the true solution. Bessel functions, elliptic integrals, and other special functions are undiscoverable unless added to the basis.

## 6 Future Work

Several directions extend the framework presented here.

**Three-dimensional Navier–Stokes solutions.** Exact 3D Navier–Stokes solutions—Beltrami flows, the Arnold–Beltrami–Childress (ABC) flow—have velocity fields that are separable products of trigonometric functions in $x$, $y$, $z$. These lie within the COMPOSEHEAD hypothesis class and would directly test scalability to higher-dimensional domains.

**Physics-informed discovery without training data.** A natural next step is to train using only the PDE residual, boundary conditions, and initial conditions—in the spirit of physics-informed neural networks but with a symbolic ansatz. The differentiation closure of the EML basis makes this extension particularly natural, since PDE residuals remain within the representable function class.

**Non-separable and nested compositions.** Two strategies may extend the method beyond separability: (i) mixed terms $f(ax + by)$ coupling coordinate variables, and (ii) nested compositions $f \circ g$ within each factor. Both enlarge the hypothesis class at the cost of combinatorial complexity.

**Automated basis selection.** An adaptive approach that begins with a small library and introduces additional primitives based on residual analysis could reduce the combinatorial burden when the relevant function class is unknown a priori.

**Multi-term scaling.** The brute-force pair search scales as $|\mathcal{F}|^{2K}$ for two-term sums, which is tractable for small $K$ and $|\mathcal{F}|$ but becomes prohibitive for higher-order decompositions. Principled strategies for multi-term discovery—greedy sequential fitting with gauge-aware normalization, or structured sparsity over a large two-term dictionary—remain open problems.

**Scaling to higher-dimensional PDEs.** PDEs in kinetic theory, finance, and quantum mechanics may involve five or more variables. Algorithmic innovations—sparsity-promoting regularization, hierarchical decomposition, or greedy term selection—will be needed to manage the exponential growth of the candidate term count.

**Integration with existing PDE solver libraries.** The COMPOSEHEAD could serve as a post-processing module for numerical solvers, accepting simulation data and returning symbolic expressions. Hybrid workflows alternating between numerical solution and symbolic identification may prove effective for parameter studies.

## 7 Conclusion

We have presented the COMPOSEHEAD, a pipeline for discovering closed-form solutions to partial differential equations from data. The two core practical contributions are: (i) a normalization pipeline that resolves the gauge freedom inherent in parameterizations of elementary functions, collapsing families of equivalent parameter settings to canonical forms amenable to coefficient snapping; and (ii) a single-term greedy search strategy that prevents signal splitting and ensures each discovered term corresponds to a complete, interpretable component of the solution. These components enable the transition from a continuously parameterized model to an exact symbolic expression. The EML universal basis provides the theoretical foundation, supplying a principled justification for the primitive function library and guaranteeing differentiation closure so that PDE residuals remain within the representable function class.

We demonstrated the method on the two-dimensional incompressible Navier–Stokes equations, recovering the Taylor–Green vortex solution $u = \sin(x)\cos(y)\exp(-t/5)$, $v = -\cos(x)\sin(y)\exp(-t/5)$ at machine precision with pointwise errors below $10^{-6}$ after normalization and coefficient snapping. The resulting expressions are genuine symbolic formulas that can be analytically differentiated, substituted into the governing equations, and verified to satisfy the Navier–Stokes system exactly.

The method fills a genuine gap. General symbolic regression methods Cranmer (2023); Udrescu and Tegmark (2020) struggle with multi-variable separable products; physics-informed neural networks produce weight matrices rather than formulas. By pairing a structured separable ansatz with gauge-resolving normalization, the COMPOSEHEAD provides an automatic path from data to exact symbolic PDE solutions—a capability that, to our knowledge, no existing tool offers.

We release `torch-eml` as an open-source PyTorch library at `https://github.com/gaius050526/torch-eml`, providing verified EML primitive implementations, the COMPOSEHEAD architecture, and the normalization and snapping pipeline described in this work.

While the current method is limited to separable solutions in two spatial dimensions and requires the primitive basis to contain the relevant functions, it establishes a foundation for symbolic PDE solving that occupies a distinctive position between neural solvers and classical analytical methods. Neural solvers offer generality at the cost of interpretability; analytical methods offer exactness but require human ingenuity. The approach presented here automates the discovery of exact, interpretable solutions within a structured function class, bridging the gap between these two paradigms.

## References


S. L. Brunton, J. L. Proctor, and J. N. Kutz. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15):3932–3937, 2016.

M. Cranmer. Interpretable machine learning for science with PySR and SymbolicRegression.jl. *arXiv preprint arXiv:2305.01582*, 2023.

A. Ipek. On elementary function representations via EML. *arXiv preprint arXiv:2604.13871*, 2024. Withdrawn by author citing "fundamental limitation in EML method".

A. Odrzywołek. The exp-minus-log function: A universal basis for computable functions. *arXiv preprint arXiv:2603.21852*, 2026.

A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 2019.

M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

J. Smith. On the limitations of the EML function for algebraic equations. *arXiv preprint*, 2024. Showed EML cannot express roots of generic quintic polynomials due to non-solvable monodromy groups.

G. I. Taylor and A. E. Green. Mechanism of the production of small eddies from large ones. *Proceedings of the Royal Society of London. Series A – Mathematical and Physical Sciences*, 158(895):499–521, 1937.

S.-M. Udrescu and M. Tegmark. AI Feynman: A physics-inspired method for symbolic regression. *Science Advances*, 6(16):eaay2631, 2020.

# A Algebraic Proofs of EML Primitive Constructions

Throughout this appendix, we denote the base EML function as

$$\mathrm{eml}(x, y) \;=\; \exp(x) - \ln(y), \tag{29}$$

where $\exp$ and $\ln$ carry their standard definitions over $\mathbb{R}$ (and, where noted, their principal-branch extensions over $\mathbb{C}$). We prove that each elementary primitive required by our compositional framework can be recovered from finitely many nested applications of $\mathrm{eml}$.

## A.1 Exponential

**Proposition A.1.** *For all $x \in \mathbb{R}$,*

$$\exp(x) \;=\; \mathrm{eml}(x,\, 1).$$

*Proof.* By direct substitution into the definition,

$$\mathrm{eml}(x,\, 1) \;=\; \exp(x) - \ln(1) \;=\; \exp(x) - 0 \;=\; \exp(x). \qquad \square$$

## A.2 Natural Logarithm

**Proposition A.2.** *For all $z > 0$,*

$$\ln(z) \;=\; \mathrm{eml}\Big(1,\, \mathrm{eml}\big(\mathrm{eml}(1,\, z),\, 1\big)\Big).$$

*The construction has depth three in nested EML operations.*

*Proof.* We evaluate the expression from the innermost call outward.

*Step 1.* Compute the innermost application:

$$\mathrm{eml}(1,\, z) \;=\; \exp(1) - \ln(z) \;=\; e - \ln(z).$$

*Step 2.* Substitute the result of Step 1 into the second position:

$$\mathrm{eml}\big(e - \ln(z),\, 1\big) \;=\; \exp\big(e - \ln(z)\big) - \ln(1) \;=\; \exp\big(e - \ln(z)\big).$$

*Step 3.* Apply the outermost call:

$$\mathrm{eml}\Big(1,\, \exp\big(e - \ln(z)\big)\Big) \;=\; \exp(1) - \ln\Big(\exp\big(e - \ln(z)\big)\Big) \;=\; e - \big(e - \ln(z)\big) \;=\; \ln(z).$$

The penultimate equality uses the identity $\ln(\exp(u)) = u$ for $u \in \mathbb{R}$. $\square$

### A.3 Sine (via Complex Extension)

**Proposition A.3.** *For all $x \in \mathbb{R}$,*

$$\sin(x) \;=\; \mathrm{Im}\big(\mathrm{eml}(ix,\,1)\big),$$

*where $i = \sqrt{-1}$ and* eml *is extended to complex arguments by employing the standard complex exponential and the principal branch of the complex logarithm.*

*Proof.* By definition,

$$\mathrm{eml}(ix,\,1) \;=\; \exp(ix) - \ln(1) \;=\; \exp(ix).$$

Euler's formula gives $\exp(ix) = \cos(x) + i\sin(x)$. Taking the imaginary part yields

$$\mathrm{Im}\big(\mathrm{eml}(ix,\,1)\big) \;=\; \mathrm{Im}\big(\cos(x) + i\sin(x)\big) \;=\; \sin(x). \qquad \square$$

**Remark A.1.** *The extension of* eml *to complex arguments is well-defined:* $\exp\colon \mathbb{C} \to \mathbb{C} \setminus \{0\}$ *is entire, and* $\ln$ *is holomorphic on $\mathbb{C} \setminus (-\infty, 0]$ under its principal branch. Since the constructions in Propositions A.3 and A.4 evaluate* $\ln$ *only at $y = 1$, no branch-cut ambiguities arise.*

### A.4 Cosine (via Complex Extension)

**Proposition A.4.** *For all $x \in \mathbb{R}$,*

$$\cos(x) \;=\; \mathrm{Re}\big(\mathrm{eml}(ix,\,1)\big).$$

*Proof.* By the same reduction as in Proposition A.3,

$$\mathrm{Re}\big(\mathrm{eml}(ix,\,1)\big) \;=\; \mathrm{Re}\big(\exp(ix)\big) \;=\; \mathrm{Re}\big(\cos(x) + i\sin(x)\big) \;=\; \cos(x). \qquad \square$$

### A.5 Pi

**Proposition A.5.** *The constant $\pi$ is recoverable from EML as*

$$\pi \;=\; \mathrm{Im}\Big(\mathrm{eml}\Big(1,\,\mathrm{eml}\big(\mathrm{eml}(1,\,-1),\,1\big)\Big)\Big).$$

*Proof.* We evaluate the nested expression under the complex extension of eml, using the principal branch $\ln(-1) = i\pi$.

*Step 1.* Innermost application:

$$\mathrm{eml}(1,\,-1) \;=\; \exp(1) - \ln(-1) \;=\; e - i\pi.$$

*Step 2.* Intermediate application:

$$\mathrm{eml}(e - i\pi,\,1) \;=\; \exp(e - i\pi) - \ln(1) \;=\; \exp(e - i\pi).$$

*Step 3.* Outermost application:

$$\mathrm{eml}\Big(1,\,\exp(e - i\pi)\Big) \;=\; \exp(1) - \ln\Big(\exp(e - i\pi)\Big) \;=\; e - (e - i\pi) \;=\; i\pi.$$

Taking the imaginary part gives $\mathrm{Im}(i\pi) = \pi$. $\square$

**Remark A.2.** *Observe that the algebraic structure of this construction is identical to that of Proposition A.2 (the logarithm), instantiated at $z = -1$. The result $\ln(-1) = i\pi$ then isolates $\pi$ as an imaginary part.*

### A.6 Differentiation Closure

**Proposition A.6.** *The class of functions expressible as finite compositions of* eml *is closed under partial differentiation with respect to either argument.*

*Proof.* We compute the partial derivatives of $\mathrm{eml}$ directly:

$$\frac{\partial}{\partial x}\,\mathrm{eml}(x,\,y) \;=\; \frac{\partial}{\partial x}\big[\exp(x) - \ln(y)\big] \;=\; \exp(x), \tag{30}$$

$$\frac{\partial}{\partial y}\,\mathrm{eml}(x,\,y) \;=\; \frac{\partial}{\partial y}\big[\exp(x) - \ln(y)\big] \;=\; -\frac{1}{y}. \tag{31}$$

It remains to show that both $\exp(x)$ and $-1/y$ lie within the EML function class.

1. **Exponential.** By Proposition A.1, $\exp(x) = \mathrm{eml}(x, 1)$.

2. **Negated reciprocal.** We note that $-1/y$ can be obtained by composing EML primitives with elementary arithmetic operations (addition, subtraction, and scalar multiplication) that are themselves constructible from EML. Specifically, $1/y = \exp(-\ln(y))$, and both $\exp$ and $\ln$ are available by Propositions A.1 and A.2. Negation follows from $-u = \mathrm{eml}(0, \mathrm{eml}(u, 1)) - 1$, since $\mathrm{eml}(0, \mathrm{eml}(u, 1)) = \exp(0) - \ln(\exp(u)) = 1 - u$.

Since partial differentiation of an EML expression produces functions that remain within the EML class, arbitrary compositions of $\mathrm{eml}$ are closed under differentiation by the chain rule. This closure property is essential for physics-informed applications in which the network must represent both a solution $u$ and its partial derivatives $\partial_t u$, $\partial_x u$, $\partial_{xx} u$, etc., within the same function class. □

### A.7 Numerical Verification

All constructions presented in Sections A.1–A.6 are numerically verified in the `torch-eml` library. The function `verify_constructions()` evaluates each EML composition and compares the result against the corresponding PyTorch reference implementation (`torch.exp`, `torch.log`, `torch.sin`, `torch.cos`, and `torch.tensor(math.pi)`).

Table 7: Numerical verification of EML primitive constructions. All absolute errors are measured against PyTorch reference values at test points $x \in \{0.5,\ 1.0,\ 2.0,\ 3.14159\}$.

| **Primitive** | **EML Construction** | **Max Abs. Error** |
|---|---|---|
| $\exp(x)$ | $\mathrm{eml}(x, 1)$ | $< 10^{-7}$ |
| $\ln(z)$ | $\mathrm{eml}(1,\, \mathrm{eml}(\mathrm{eml}(1, z), 1))$ | $< 10^{-5}$ |
| $\sin(x)$ | $\mathrm{Im}(\mathrm{eml}(ix, 1))$ | $< 10^{-7}$ |
| $\cos(x)$ | $\mathrm{Re}(\mathrm{eml}(ix, 1))$ | $< 10^{-7}$ |
| $\pi$ | $\mathrm{Im}(\mathrm{eml}(1, \mathrm{eml}(\mathrm{eml}(1, -1), 1)))$ | $< 10^{-7}$ |

The maximum error across all constructions and all test points is bounded by $10^{-5}$. The slightly elevated error for the logarithm construction reflects the accumulation of floating-point rounding across three nested EML evaluations, each involving an exponentiation step. All errors remain well within tolerances acceptable for both training and evaluation of physics-informed neural surrogates.

## B Snap Targets and Normalization

After training, learned parameters are continuous floating-point values. To recover closed-form symbolic expressions, each parameter is rounded ("snapped") to the nearest value in a predefined table of mathematically meaningful targets. Table 8 enumerates the complete set of snap targets used throughout this work.

Given a learned parameter $\hat{\theta}$, snapping selects $\theta^* = \arg\min_{\theta \in \mathcal{T}} |\hat{\theta} - \theta|$, where $\mathcal{T}$ is the set of targets above, subject to the constraint that $|\hat{\theta} - \theta^*| < \epsilon_{\mathrm{snap}}$. If no target lies within the tolerance $\epsilon_{\mathrm{snap}}$, the raw floating-point value is retained.

Table 8: Complete table of snap targets used for rounding learned parameters to clean symbolic values.

| Target Value | Numeric Value | Label |
|---|---|---|
| $0$ | 0.000000 | zero |
| $+1$ | 1.000000 | one |
| $-1$ | $-1.000000$ | neg one |
| $+2$ | 2.000000 | two |
| $-2$ | $-2.000000$ | neg two |
| $+3$ | 3.000000 | three |
| $-3$ | $-3.000000$ | neg three |
| $+\frac{1}{2}$ | 0.500000 | half |
| $-\frac{1}{2}$ | $-0.500000$ | neg half |
| $+\frac{1}{3}$ | 0.333333 | third |
| $-\frac{1}{3}$ | $-0.333333$ | neg third |
| $+\frac{2}{3}$ | 0.666667 | two-thirds |
| $-\frac{2}{3}$ | $-0.666667$ | neg two-thirds |
| $+\frac{1}{4}$ | 0.250000 | quarter |
| $-\frac{1}{4}$ | $-0.250000$ | neg quarter |
| $+\frac{1}{5}$ | 0.200000 | fifth |
| $-\frac{1}{5}$ | $-0.200000$ | neg fifth |
| $+\frac{3}{2}$ | 1.500000 | three-halves |
| $-\frac{3}{2}$ | $-1.500000$ | neg three-halves |
| $+\pi$ | 3.141593 | pi |
| $-\pi$ | $-3.141593$ | neg pi |
| $+\frac{\pi}{2}$ | 1.570796 | pi/2 |
| $-\frac{\pi}{2}$ | $-1.570796$ | neg pi/2 |
| $+\frac{\pi}{4}$ | 0.785398 | pi/4 |
| $-\frac{\pi}{4}$ | $-0.785398$ | neg pi/4 |
| $+e$ | 2.718282 | euler |
| $-e$ | $-2.718282$ | neg euler |
| $+\sqrt{2}$ | 1.414214 | sqrt2 |
| $-\sqrt{2}$ | $-1.414214$ | neg sqrt2 |
| $+\ln 2$ | 0.693147 | ln2 |
| $-\ln 2$ | $-0.693147$ | neg ln2 |

### B.1 Normalization of `SeparableTerm`

The parameterization of a `SeparableTerm` as a product

$$c \prod_k f_k(a_k x_k + b_k), \tag{32}$$

where each $f_k \in \{\exp, \sin, \cos, \mathrm{id}\}$, possesses *gauge freedom*: multiple distinct parameter settings yield the same function. For example,

$$c \cdot \exp(ax + b) \;=\; \left[c \cdot e^{b}\right] \cdot \exp(ax), \tag{33}$$

so the coefficient $c$ and bias $b$ can trade off arbitrarily without changing the represented function. If left unnormalized, the optimizer may converge to any point along this gauge orbit, placing the learned parameters far from the snap targets and causing snapping to produce incorrect symbolic expressions. Normalization breaks this symmetry by imposing canonical constraints before snapping is applied.

The normalization algorithm proceeds in three steps:

**Step 1: Exponential bias absorption.** For each exponential factor $\exp(a_k x_k + b_k)$ in the term, the bias is absorbed into the coefficient:

$$c \leftarrow c \cdot e^{b_k}, \qquad b_k \leftarrow 0. \tag{34}$$

After this step, every exponential factor has the form $\exp(a_k x_k)$ and the coefficient $c$ carries the full scale.

**Step 2: Trigonometric bias reduction.** For each sinusoidal factor $\sin(a_k x_k + b_k)$ or $\cos(a_k x_k + b_k)$, the bias $b_k$ is reduced to the principal interval $[-\pi, \pi]$ via modular arithmetic:

$$b_k \leftarrow \big((b_k + \pi) \bmod 2\pi\big) - \pi. \tag{35}$$

If, after reduction, $|b_k| > \frac{\pi}{2} + \varepsilon$, a half-period shift is applied. Using the identity $\sin(x + \pi) = -\sin(x)$ (and analogously for cosine), we set

$$b_k \leftarrow b_k - \operatorname{sign}(b_k)\,\pi, \qquad c \leftarrow -c. \tag{36}$$

This ensures all trigonometric biases lie in $[-\frac{\pi}{2} - \varepsilon,\ \frac{\pi}{2} + \varepsilon]$, concentrating them near the small set of snap targets $\{0, \pm\frac{\pi}{4}, \pm\frac{\pi}{2}\}$.

**Step 3: Sign canonicalization via sine parity.** If the overall coefficient $c$ is negative and the term contains at least one sine factor $\sin(a_j x_j + b_j)$, the sign is absorbed using the odd-symmetry identity $\sin(-z) = -\sin(z)$:

$$a_j \leftarrow -a_j, \qquad b_j \leftarrow -b_j, \qquad c \leftarrow -c. \tag{37}$$

This removes a residual sign ambiguity, ensuring that the coefficient $c$ is non-negative whenever a sine factor is present.

Together, these three steps place the parameters of each `SeparableTerm` into a canonical form in which every degree of freedom corresponds to a genuinely independent quantity, making the subsequent snapping procedure both reliable and unambiguous.

# C Reproducibility

We provide full details necessary to reproduce every experiment reported in this paper.

## C.1 Code Availability

All source code is publicly available at `https://github.com/gaius050526/torch-eml` under the MIT license. The library, `torch-eml` v0.1.0, is implemented entirely in PyTorch and can be installed directly from the repository.

## C.2 Library Structure

The codebase is organized into the following key modules:

- `torch_eml.primitives` — Verified EML constructions providing the atomic building blocks of the framework: `EMLExp`, `EMLLn`, `EMLSin`, `EMLCos`, and `EMLPi`.
- `torch_eml.compose` — Higher-level architectures for composing primitives into multi-variable expressions, including `ComposeHead`, `SeparableTerm`, and `AxisTerm`.
- `torch_eml.symbolic` — Symbolic extraction and snapping utilities that convert trained network parameters into closed-form SymPy expressions.

### C.3 Dependencies

- PyTorch $\geq 2.0$
- SymPy $\geq 1.12$

No additional dependencies beyond standard Python scientific libraries are required.

### C.4 Hardware

All experiments reported in this paper were run on a single CPU (Apple M-series). No GPU is required. The complete Taylor–Green vortex search, including both the search and fine-tuning phases, completes in approximately two minutes on this hardware.

### C.5 Random Seed

All experiments use a fixed random seed for reproducibility:

```
torch.manual_seed(42)
```

### C.6 Hyperparameters for the Taylor–Green Experiment

Table 9 summarizes the hyperparameters used for the Taylor–Green vortex experiment.

Table 9: Hyperparameters for the Taylor–Green vortex experiment.

| **Parameter** | **Value** |
|---|---|
| Training data points | 3000 |
| Spatial domain | $x, y \sim \mathcal{U}[0,\, 2\pi]$ |
| Temporal domain | $t \sim \mathcal{U}[0,\, 1]$ |
| Function basis | $\{\sin,\, \cos,\, \exp\}$ |
| *Search phase* | |
| Steps | 2000 per candidate |
| Optimizer | Adam |
| Learning rate | 0.01 |
| Gradient clipping | 1.0 |
| *Fine-tune phase* | |
| Steps | 5000 |
| Optimizer | Adam |
| Learning rate | 0.005 |
| Gradient clipping | 1.0 |
| Snap tolerance | 0.05 |

### C.7 Test Suite

The repository includes a comprehensive test suite comprising 117 tests that cover all modules, including primitive correctness, composition semantics, symbolic extraction, and end-to-end integration. All tests pass under the released version of the library.